

MARLET: Multi-Agent Retrieval with Lightweight Evidence and Task-State Control

Anonymous Authors

Abstract—Large language models can generate fluent responses, but reliable domain assistance requires grounding in external evidence, visible user or task state, and explicit answer criteria. Classical retrieval-augmented generation (RAG) usually treats grounding as a fixed retrieve-then-read step in which a query is matched against a corpus and the generator reads the returned passages. This control flow is rigid when the same surface question requires different evidence, response structure, or evaluation criteria across situations. We propose MARLET (Multi-Agent Retrieval with Lightweight Evidence and Task-State Control), a configurable multi-agent retrieval framework that treats RAG as structured prompt construction rather than query-only passage lookup. A supervisory router identifies the response regime; LLM-backed planning, state diagnosis, criteria analysis, and conditional retrieval prepare typed context blocks; and all intermediate outputs are reassembled into one augmented prompt for a fixed generator and bounded critic. The reusable mechanism is domain-general: only cue lexicons, default criteria, and corpus/index contents are domain configuration. Controlled validation in educational tutoring under matched backbone, corpus, reranker, and response cap shows that MARLET improves quality proxies while reducing token use, with additional latency and model-call reductions in some settings.

I. Introduction

Large language models (LLMs) are increasingly used as domain assistants because they can generate explanations, diagnose errors, and provide feedback. However, a useful assistant cannot rely only on parametric knowledge. Knowledge-grounded dialogue has long shown the value of external evidence [1]; domain assistance similarly needs grounding in manuals, policies, evaluation criteria, worked examples, user profiles, or task records to avoid fluent but misaligned guidance. Educational tutoring is a demanding instantiation of this broader problem [2], [3].

Classical retrieval-augmented generation (RAG) has therefore become the default grounding mechanism [4]. In classical RAG, the query is embedded, the top- k most similar passages are retrieved, and the LLM generates a response conditioned on those passages. Thus, relevance depends heavily on retrieval quality and query representation [5].

Domain assistance is more complex than factual question answering. A user’s question is often only a partial signal: the appropriate response may depend on expertise, goal, task history, constraints, prior confusion, and evaluation criteria. We call these visible attributes

the user’s personalized task state. In education, this state becomes learner level and misconceptions; in technical support, it becomes device state and attempted fixes; in policy or legal assistance, it becomes jurisdiction, role, and applicable rules. Therefore, retrieval must find evidence that is appropriate for the task state, not merely topical.

Classical RAG can include explicit state text when supplied, but appending user state, dialogue, criteria, and task constraints forces one query embedding to compress topic, level of detail, intent, and response requirements. As constraints grow, search intent can blur. The issue is therefore how a response system should organize constraints before retrieval and generation.

Therefore, we introduce MARLET (Multi-Agent Retrieval with Lightweight Evidence and Task-State Control). Although instantiated for educational validation, MARLET is a configurable RAG controller: instead of letting the retriever act as the top-level controller, a supervisory router identifies the response regime and decides whether retrieval is needed. A planner scopes search intent, a state diagnoser summarizes visible user or task state, and a criteria module states answer requirements; these modules run in parallel when possible, then are reassembled with retrieved evidence into one generator prompt. The framework integrates selective retrieval, tool-using agents, and state-aware criteria modeling into one context-construction mechanism.

The experiments validate the framework rather than present a model leaderboard. We compare with classical retrieve-then-read RAG and Agentic RAG under shared backbone, corpus, reranker, response cap, generator, and critic. The educational instantiation shows that the same generator scores higher when fed context assembled by MARLET, with lower resource cost in several settings.

Contributions. (i) We propose MARLET, a configurable multi-agent retrieval framework with a supervisory router, planner, state diagnoser, criteria module, conditional retriever, generator, and critic. (ii) We formalize the router and retrieval gate as transparent, training-free controls over task state, answer criteria, and external evidence. (iii) We instantiate and validate the framework against classical RAG and Agentic RAG baselines on two educational benchmarks under shared backbone, corpus, reranker, and response cap.

II. Related Work

Classical RAG fixes retrieval as the first controller: the query is matched to passages, and generation conditions on the returned evidence [4]. Selective and corrective RAG methods improve this controller. Self-RAG learns when to retrieve, generate, and critique; CRAG evaluates retrieved documents and attempts correction when retrieval is unreliable; Adaptive-RAG and TARG vary retrieval behavior by query complexity or retrieval utility [6]–[9]. RAG diagnostics further emphasize separating retrieval, ranking, and generation errors [10]. These methods address whether passages retrieved for a query are necessary, sufficient, or reliable, but they do not explicitly organize user/task state, criteria, and evidence as separate prompt constraints before retrieval.

Agentic RAG and multi-agent LLM systems move beyond a single retrieve-then-read step. ReAct, AutoGen, CAMEL, and MetaGPT show that tool use and role decomposition can improve complex task execution [11]–[14]. MA-RAG uses multiple agents for query disambiguation, evidence extraction, and answer synthesis on ambiguous or multi-hop QA tasks [15]. These systems demonstrate coordination benefits, but their roles are usually tied to task execution or ambiguous question answering rather than a reusable controller that chooses among planning, criteria checking, state adaptation, and evidence grounding before final prompt assembly.

Recent educational work shows one high-stakes setting where state-aware grounding matters. AgentTutor builds multi-turn personalized learning with learner assessment and memory [3]; adaptivity studies show that LLM tutors are sensitive to omitted student-context features [16]; and AI-tutor evaluation emphasizes pedagogy, mistake diagnosis, guidance quality, and learner support [2], [17]–[20]. These works motivate education as a validation domain, but they do not formulate a domain-configurable retrieval controller for deciding when to retrieve, what evidence to retrieve, and how much prompt space to allocate to evidence versus state and criteria.

III. Methodology

MARLET turns one domain request into a grounded response. It is a RAG controller rather than a set of independent answer agents: every intermediate output is converted into context for one final LLM prompt. Its design integrates three domain-general principles: route before retrieval, exchange typed records rather than free-form chat, and retrieve only when the response brief needs external evidence. In education, the final generator is a tutor; in another domain, the same controller could drive a legal assistant, clinical summarizer, or technical support agent by replacing cue lexicons, default criteria, and the corpus. The flow is unchanged: standardize the entry, score routing cues, run active LLM-backed specialists in parallel, merge their outputs with retrieved evidence, serialize one augmented prompt, and run one critic revision if needed.

Algorithm 1: MARLET Inference Process

Require: entry $x = (q, o, c, h, r, v)$, caps K, Q, L, N , index $\mathcal{I}$, thresholds τ .

Return: answer y and internal grounding fields $(R, D, \text{citations})$.

```

1:  $b \leftarrow \text{concat}(q, c, r, h)$ ;  $s \leftarrow \text{CueScores}(b)$ 
2:  $(R, G) \leftarrow \text{Route}(s, r, v, \tau)$ 
3: parallel do
4:    $p = (\text{strategy}, P) \leftarrow \text{Planner}(x, R, G)$ 
5:    $\ell \leftarrow \text{State}(q, h)$ 
6:   if  $r \neq \emptyset$  or  $R = \text{CF}$  then  $u \leftarrow \text{Criteria}(x, R)$  else  $u \leftarrow \text{DefaultCriteria}()$ 
7: end parallel
8:  $P \leftarrow \text{cap}(P, K)$ ;  $D \leftarrow \emptyset$ 
9: if  $G = 1$  then
10:   $D \leftarrow \text{Pack}(\text{Rerank}(\text{Dedup}(\text{Search}(\mathcal{I}, P)), q), Q, L)$ 
11: end if
12:  $y_0 \leftarrow A_{\text{gen}}(\text{Prompt}(q, o, h, p, \ell, u, D, c), N)$ 
13: if  $D = \emptyset$  and  $s_{\text{ev}} \geq \tau_2$  then
14:   $D \leftarrow \text{Pack}(\text{Rerank}(\text{Search}(\mathcal{I}, [q]), q), Q, L)$ 
15:   $y_0 \leftarrow A_{\text{gen}}(\text{Prompt}(q, o, h, p, \ell, u, D, c), N)$ 
16: end if
17:  $y \leftarrow \text{CriticOnce}(y_0, q, h, p, \ell, u, D, r)$ 
18: return  $y, (R, D, \text{ids}(D))$ 

```

A. Input Entry and Inference Caps

For one sample, MARLET first standardizes all visible fields into

$$x = (q, o, c, h, r, v). \quad (1)$$

Here, q is the user question or task prompt; o is an optional answer-option list; c is optional inline context such as a policy excerpt, manual page, or chapter passage; h is recent interaction history; r is an optional criteria list; and v is an optional image list. Empty fields are allowed, so the same representation covers factual questions, assessment requests, feedback requests, and planning requests.

The only user-facing output is the final answer y . Internally, MARLET keeps a route record, an evidence bundle D with packed retrieved chunks, and citation identifiers linking chunks in D to their sources. Planner output, state summary, criteria summary, and D are assembled as prompt blocks before the LLM generator produces y . Fixed inference caps keep the process bounded: K caps LLM planner and retrieval-query outputs, Q caps packed chunks/tool calls, L caps packed-evidence length, and N caps response tokens.

B. Supervisor Routing

The supervisor begins with a routing blob:

$$b(x) = \text{concat}(q, c, r, h), \quad (2)$$

where $\text{concat}(\cdot)$ means that the question, inline context, criteria text, and interaction turns are joined into one normalized string. Images remain available to the final generator, but they are not converted into the textual routing score.

The router uses four cue families $j \in \{\text{ev}, \text{cr}, \text{pl}, \text{ad}\}$ for evidence, criteria, planning, and adaptation. These are generic response-construction needs rather than benchmark or domain labels. Evidence cues ask for source-grounded support; criteria cues ask the system to judge an answer against explicit requirements; planning cues ask for an ordered action path; and adaptation cues indicate that the response should be shaped by

user state, expertise, or prior interaction. Each family has a keyword set W_j . Evidence cues include source, document, manual, and verification; criteria cues include score, criteria, and comment; planning cues include plan, sequence, and next steps; adaptation cues include user, beginner, expert, confused, and explain. Let $M_j(x)$ be the number of cues from W_j that appear in $b(x)$. The cue score is

$$s_j(x) = \frac{M_j(x)}{|W_j|}, \quad (3)$$

where $|W_j|$ is the number of cues in family j , so $s_j(x)$ is the fraction of that cue family observed in the routing blob. The thresholds are fixed router hyperparameters, not probabilities or learned weights. We choose them in cue-count units so τ_{mid} requires a repeated cue pattern rather than one accidental word, τ_{plan} is stricter because a false planning route changes the whole answer structure, and τ_2 permits retry only for strong evidence cues. Porting the framework to a new domain mainly means replacing W_j and default criteria while keeping the router equation and control flow.

After cue scoring, the supervisor chooses a response regime. A regime is not a dataset label; it is a generic prompt-priority decision made before retrieval and generation. The intuition is to ask what missing constraint would most likely make the final answer wrong. If the user needs an ordered path, planning (PL) reserves space for sequence and dependencies. If the task asks for judgment, criteria feedback (CF) reserves space for evaluation rules and scoring logic. If the user’s state changes how the same content should be presented, adaptive response (AR) reserves space for user state. If correctness depends mainly on outside material, evidence-grounded response (EG) reserves space for source evidence. Because these needs can overlap, $R(x)$ is selected by the priority rule

$$R(x) = \begin{cases} \text{PL}, & s_{\text{pl}} \geq s_{\text{cr}}, s_{\text{pl}} \geq s_{\text{ad}}, s_{\text{pl}} \geq \tau_{\text{plan}}, \\ \text{CF}, & s_{\text{cr}} \geq s_{\text{ad}}, s_{\text{cr}} \geq \tau_{\text{mid}}, \\ \text{AR}, & s_{\text{ad}} \geq \tau_{\text{mid}}, \\ \text{EG}, & \text{otherwise.} \end{cases} \quad (4)$$

The order reflects generic constraint strength. Sequence changes the whole response, explicit criteria constrain evaluation, user state shapes delivery, and evidence is the default grounding route when the request does not clearly require one of the first three prompt shapes.

Next, the supervisor uses a retrieval gate: a binary control for whether external evidence enters the final prompt. Inspired by selective-retrieval research, MARLET integrates this idea as a prompt-allocation control that opens only when outside grounding is needed [6], [8], [9]:

$$G(x) = \begin{cases} 1, & s_{\text{ev}} \geq \tau_{\text{mid}} \text{ or } R(x) = \text{EG}, \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

Thus $G(x) = 1$ runs retrieval; $G(x) = 0$ builds the same prompt without external chunks. A stricter retry gate runs one raw-question pass only when the first bundle is

empty:

$$G_2(x) = \begin{cases} 1, & D_0 \text{ is empty and } s_{\text{ev}} \geq \tau_2, \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

where D_0 is the evidence bundle from the first retrieval attempt.

C. Parallel Agent Brief Construction

The router and gates above decide what the final prompt should prioritize; MARLET then builds a shared response brief. Inspired by tool-using and multi-agent LLM systems [11], [12], the specialist agents do not draft competing answers. Instead, they produce complementary control fields for one generator: the planner narrows the operating strategy and search intent, the state diagnoser records visible user/task conditions, and the criteria module states what the answer must satisfy. These LLM-backed context builders run in parallel:

$$(p, \ell, u) = (A_{\text{plan}}(x, R, G), A_{\text{state}}(q, h), A_{\text{criteria}}(x, R)). \quad (7)$$

Here p contains a short strategy and up to K retrieval queries, ℓ summarizes visible task state, and u stores explicit or default answer criteria. This separation prevents the final generator from having to recover plan, state, and criteria from one overloaded query embedding.

The planner receives x , R , and G . Its strategy indicates whether the response should be direct, criteria-driven, scaffolded, or evidence-heavy; its query list P is intentionally shorter than the full prompt so retrieval is driven by searchable terms rather than conversational noise. A deterministic key-term fallback is used only if the LLM endpoint fails or returns malformed JSON. The state diagnoser uses only q and h to summarize visible conditions such as expertise level, goal, confusion, prior attempts, or requested style, reflecting evidence that omitted user-context features affect generated assistance [3], [16]. It is not a persistent user profile and does not infer hidden demographics. The criteria module preserves explicit requirements from r when available; otherwise it supplies a small checklist for correctness, clarity, evidence use when available, and actionable next steps. The coordinator then merges these typed fields as (R, G, p, ℓ, u) .

D. Conditional Retrieval and Evidence Packing

MARLET contains retrieval, but retrieval is not the top-level controller and does not require a traditional dense vector database. When $G(x) = 1$, an LLM-backed retrieval agent refines the planner’s search intents into up to K compact queries, and each query q_i searches a local hybrid text index. For a query q_i and candidate chunk d , the base retrieval score is

$$\text{base}(q_i, d) = w_{\text{lex}} \text{lex} + w_{\text{char}} \text{char} + w_{\text{svd}} \text{svd} + \beta. \quad (8)$$

Here lex is word TF-IDF similarity, char is character 3–5 gram TF-IDF similarity, svd is truncated-SVD similarity, β is a small title/source boost, and the w

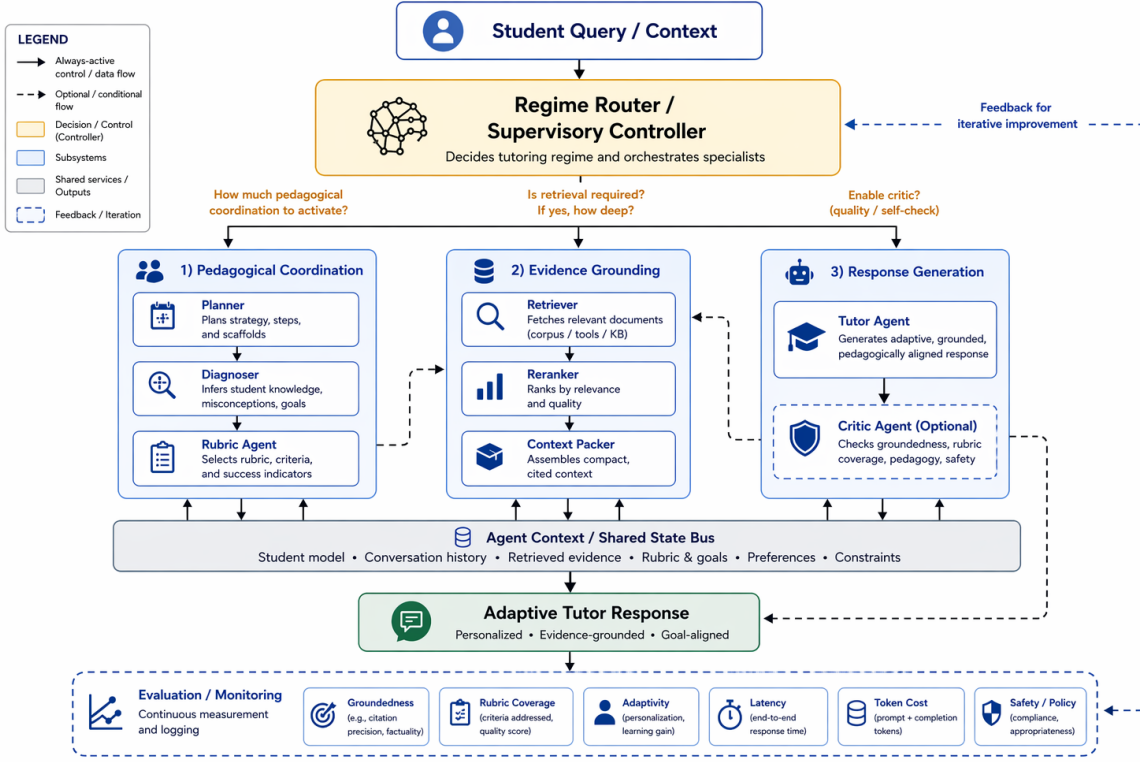


Fig. 1. Main framework of MARLET. The router scores the entry, activates specialist modules, and decides whether retrieval is needed; specialist outputs and retrieved evidence are then assembled into one augmented generator prompt.

terms are fixed retriever weights. Candidates from all planner queries are merged, and duplicate chunks keep their highest score.

The reranker then scores each candidate against the original question:

$$\text{rank}(q, d) = \alpha_{\text{base}} \text{base} + \alpha_{\text{tok}} o_{\text{tok}} + \alpha_{\text{title}} o_{\text{title}} + \alpha_{\text{phr}} m_{\text{phr}} + \alpha_{\text{num}} o_{\text{num}} + \alpha_{\text{len}} b_{\text{len}}. \quad (9)$$

Here o_{tok} is token overlap with q , o_{title} is title-token overlap, m_{phr} is an exact phrase-match flag, o_{num} is numeric overlap, b_{len} rewards moderate chunk length, and the α terms are fixed reranker weights. The state summary ℓ affects retrieval through the route and planner queries, not these weights.

Evidence packing integrates the relevance-diversity tradeoff used in MMR-style selection [21]: the context should be useful, but repeated chunks should not consume the limited evidence space. For each candidate chunk d , MARLET computes

$$\text{pack}(d) = \lambda \text{rank}(q, d) - (1 - \lambda) \text{repeat}(d). \quad (10)$$

Here $\text{repeat}(d)$ is the largest sentence overlap between d and selected chunks, and λ controls the relevance-diversity balance. The packer adds the remaining chunk with the highest $\text{pack}(d)$ score until Q chunks or the L character limit is reached, forming evidence bundle D .

E. Generation and Critic Revision

This is where retrieval and agent outputs converge: (R, G, p, ℓ, u) supplies route, plan, state, and criteria

fields, while D supplies compact cited evidence. The retriever does not answer; its chunks become an evidence block. The final prompt contains task fields, response brief, evidence or inline context, and output requirements. The generator is the LLM-driven answer writer:

$$y_0 = A_{\text{gen}}(q, o, h, p, \ell, u, D, c). \quad (11)$$

Its prompt contains q , optional answer options o , recent interaction history h , state summary ℓ , plan p , criteria summary u , and either retrieved evidence D or inline context c . The generator is instructed by the domain configuration to be correct, useful, concise but not terse, include a next step when appropriate, and cite retrieved evidence with document markers.

The critic performs one bounded revision pass. It checks for missing citation markers when D is nonempty, uncovered criteria from r , or a response style that conflicts with the inferred state. If no issue is found, $y = y_0$; otherwise the critic rewrites once using $(y_0, q, h, p, \ell, u, D, r)$ and the revised y becomes the final answer.

IV. Validation Experiments

We validate the framework on two public educational benchmarks used as domain-specific stress tests. TutorEval [2] pairs science tutoring questions with long STEM chapters and teaching key points; EduBench [18] covers diverse educational scenarios with prompts, meta-data, and reference model predictions that our adapter converts into score and rubric supervision.

The primary validation compares three context controllers under matched infrastructure, with a Qwen3.5-72B-FP8 replication checking backbone sensitivity. Classical RAG retrieves first from the raw standardized prompt; visible learner profile, student answer, dialogue, and criteria text remain available to its generator prompt, so the baseline is not deprived of task context. Its limitation is controller order: retrieval happens before specialist summaries can shape search. Agentic RAG runs the same planner, state diagnoser, and criteria module before retrieval, but retrieval is always on. MARLET keeps the same specialists and retriever, then adds supervisory routing, conditional retrieval, and retry control. We report quality and cost together because only context construction changes.

A. Implementation Details

All controllers use the same deployed Python framework, corpus, hybrid text index, reranker, generator, critic, and metric code; only context assembly differs. The planner, state diagnoser, criteria module, retrieval-query refiner, generator, and critic use the same configured backbone; deterministic fallbacks run only when an LLM response is unavailable or malformed. LLM endpoints are served through SGLang, whose runtime is designed for structured multi-call LLM programs and efficient KV-cache reuse [22]. Runs use Qwen3.5-4B and Qwen3.5-72B-FP8, reasoning budget 512, generation temperature 0.15, specialist JSON temperature 0.0, and response cap $N=900$ on benwulab-Lambda-Vector with two NVIDIA RTX 6000 Ada GPUs. We set $K=3$, $Q=4$, $L=6000$, router thresholds $(\tau_{\text{mid}}, \tau_{\text{plan}}, \tau_2) = (0.35, 0.42, 0.45)$, retriever weights $(0.58, 0.17, 0.25)$, reranker weights $(0.55, 0.20, 0.10, 0.06, 0.05, 0.04)$, and packing weight $\lambda=0.68$.

Evaluation is dataset-aware, and all quality scores are automatic proxies rather than official benchmark or human grades. Inspired by RAG evaluation work [10], we separate answer overlap, criteria fulfillment, grounding, and resource cost. Token-F1 balances reference-token coverage against extra generated tokens; rubric coverage checks whether benchmark criteria are expressed with exact or content-bearing partial matches; latency is wall-clock seconds per sample; and tokens count prompt plus completion tokens.

For EduBench, EB12D averages 12 normalized checks over scenario adaptation, factual/reasoning behavior, and pedagogical application, while EduAlign only measures extracted-score agreement with the benchmark consensus. For TutorEval, tutor-hit measures expert teaching-point coverage with full, partial, or zero credit.

B. Framework Validation

Table I reports paired 4B comparisons with bootstrap CIs and permutation p-values; Table II reports the 27B replication. Agentic RAG uses the same specialist brief as MARLET but retrieves for every sample, and its

TABLE I
4B quality and resource summary.

Data	Metric	RAG	Ours	Δ
TutorEval	Token-F1	0.112	0.115	+0.004*
	TE hit	0.938	0.957	+0.019**
	Rubric	0.919	0.944	+0.026**
	Latency (s)	51.90	49.11	-2.79
	Tokens	3975	3490	-484***
EduBench	EB12D	0.367	0.398	+0.031***
	Rubric	0.705	0.891	+0.186***
	EduAlign	0.166	0.126	-0.040**
	Latency (s)	19.83	13.90	-5.92***
	Tokens	4020	2226	-1794***

*** $p<10^{-4}$, ** $p<0.01$, * $p<0.05$. Higher is better for quality; lower is better for latency and tokens. Runtime is mean wall-clock seconds/sample on the experiment server.

TABLE II
27B quality and resource summary.

Data	Metric	RAG	Ours	Δ
TutorEval	Token-F1	0.264	0.272	+0.008*
	TE hit	0.897	0.918	+0.021*
	Rubric	0.861	0.891	+0.030*
	Latency (s)	41.47	40.65	-0.82**
	Tokens	6681	6730	+49
EduBench	EB12D	0.355	0.425	+0.070***
	Rubric	0.634	0.769	+0.135***
	EduAlign	0.541	0.526	-0.015
	Latency (s)	41.91	32.09	-9.81***
	Tokens	6105	3717	-2388***

TutorEval uses $n=400$ paired samples; EduBench uses $n=328$ unique paired rows after duplicate public IDs in the 400-example slice are collapsed. *** $p<10^{-4}$, ** $p<0.01$, * $p<0.05$. Higher is better for quality; lower is better for latency and tokens. Runtime is mean wall-clock seconds/sample on the experiment server.

matched results are being generated under the same protocol.

C. Trade-offs

For TutorEval, the 27B paired CIs are positive for Token-F1, tutor-hit, rubric coverage, and latency. For EduBench, EB12D and rubric coverage improve, latency and token use decrease, and the small classical-RAG EduAlign edge is not statistically reliable.

MARLET can use more modules than classical RAG, so resource cost must be reported. Scoped planner queries and the retrieval gate reduce token use in 4B and still reduce latency in 27B, but 27B TutorEval shows that better coverage does not always mean fewer tokens. Thus MARLET decides what grounding a request needs; it is not a universal retrieval replacement.

V. Limitations

The study validates a framework rather than a deployable assistant: evaluation is automatic and English-only, and user or task state is inferred from visible prompt/dialogue fields. Deployment requires privacy safeguards, bias checks, data governance, human oversight, and domain-specific outcome studies.

VI. Conclusion

MARLET reframes grounding as supervised context construction rather than query-only passage lookup.

With shared infrastructure, the educational instantiation improves fixed-generator quality and efficiency over classical RAG by coordinating task state, answer criteria, and conditional evidence before generation. Future work should test stronger controller baselines, additional domains, human and multi-turn outcomes, learned router calibration, profile-aware reranking, and privacy-preserving state inference.

References

- [1] E. Dinan, S. Roller, K. Shuster, A. Fan, M. Auli, and J. Weston, “Wizard of Wikipedia: Knowledge-powered conversational agents,” in *Proceedings of the International Conference on Learning Representations*, 2019. [Online]. Available: <https://arxiv.org/abs/1811.01241>
- [2] A. Chevalier, J. Geng, A. Wettig, H. Chen, S. Mizera, T. Annala, M. J. Aragon, A. R. Fanlo, S. Frieder, S. Machado, A. Prabhakar, E. Thieu, J. T. Wang, Z. Wang, X. Wu, M. Xia, W. Xia, J. Yu, J.-J. Zhu, Z. J. Ren, S. Arora, and D. Chen, “Language models as science tutors,” *arXiv:2402.11111*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.11111>
- [3] Y. Liu, Z. Song, J. Lou, C. Wu, and J. Li, “AgentTutor: Empowering personalized learning with multi-turn interactive teaching in intelligent education systems,” *arXiv:2601.04219*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.04219>
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html
- [5] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021. [Online]. Available: <https://openreview.net/forum?id=wCu6T5xFjeJ>
- [6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2310.11511>
- [7] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” *arXiv:2401.15884*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.15884>
- [8] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, “Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 7036–7050. [Online]. Available: <https://aclanthology.org/2024.naacl-long.389/>
- [9] Y. Wang, L. Wei, and H. Ling, “Retrieval as a decision: Training-free adaptive gating for efficient RAG,” *arXiv:2511.09803*, 2026. [Online]. Available: <https://arxiv.org/abs/2511.09803>
- [10] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, H. Li, Z. Zhang, B. Wang, J. Jiang, T. He, Z. Wang, P. Liu, Y. Zhang, and Z. Zhang, “RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.08067>
- [11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [12] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv:2308.08155*, 2024. [Online]. Available: <https://arxiv.org/abs/2308.08155>
- [13] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative agents for “mind” exploration of large language model society,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.17760>
- [14] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.00352>
- [15] T. Nguyen, P. Chin, and Y.-W. Tai, “MA-RAG: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning,” *arXiv:2505.20096*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.20096>
- [16] C. Borchers and T. Shou, “Can large language models match tutoring system adaptivity? A benchmarking study,” in *Artificial Intelligence in Education*. Springer Nature Switzerland, 2025, pp. 407–420. [Online]. Available: <https://arxiv.org/abs/2504.05570>
- [17] K. K. Murya, K. A. Srivatsa, K. Petukhova, and E. Kochmar, “Unifying AI tutor evaluation: An evaluation taxonomy for pedagogical ability assessment of LLM-powered AI tutors,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025. [Online]. Available: <https://aclanthology.org/2025.naacl-long.57/>
- [18] B. Xu, Y. Bai, H. Sun, Y. Lin, S. Liu, X. Liang, Y. Li, Z. Dong, J. Zhang, Y. Deng, X. Zou, Y. Gao, and H. Huang, “EduBench: A comprehensive benchmarking dataset for evaluating large language models in diverse educational scenarios,” *arXiv:2505.16160*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.16160>
- [19] J. Macina, N. Daheim, I. Hakimi, M. Kapur, I. Gurevych, and M. Sachan, “MathTutorBench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 204–221. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.11/>
- [20] R. S. Srinivasa, Z. Che, C. B. C. Zhang, D. Mares, E. Hernandez, J. Park, D. Lee, G. Mangialardi, C. Ng, E.-Y. H. Cardona, A. Gunjal, Y. He, B. Liu, and C. Xing, “TutorBench: A benchmark to assess tutoring capabilities of large language models,” *arXiv:2510.02663*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.02663>
- [21] J. Carbonell and J. Goldstein, “The use of mmr, diversity-based reranking for reordering documents and producing summaries,” in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336.
- [22] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 62 557–62 583. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html